<h2 id="%F0%9F%94%97-Live-Deployment-Link">🔗 Live Deployment Link</h2>
Live App URL: https://heart-disease-app-jes4zdm4hfczcb3vxypo7w.streamlit.app/

<h1 id="FA24-BSE-070-M.Ahmed-Waqar">FA24-BSE-070 M.Ahmed Waqar</h1><h1 id="Assignment-2">Assignment 2</h1><h3 id="Heart-Disease-Prediction-System">Heart Disease Prediction System</h3> Machine Learning Fundamentals (AIC354) – COMSATS University Islamabad, Lahore Campus

<h2 id="Table-of-Contents">Table of Contents</h2>

- Introduction and Input / Output
- Step 1: Import Libraries
- Step 2: Load Sample Data
- Step 3: Understand and Pre-process Sample Data
  - Step 3.1: Understand Sample Data
  - Step 3.2: Pre-process Sample Data (prepare the output, check missing values)
  - Step 3.3: Split Sample Data into Training Data and Testing Data
- Step 4: Feature Selection (exactly 4 input attributes)
  - Step 4.1: Rank All Attributes using Training Data only
  - Step 4.2: Select the Four Attributes
- Step 5: Numeric Representation of Input and Output (No Label Encoding)
  - Step 5.1: Check the Output
  - Step 5.2: Check the Input
- Step 6: Execute the Training Phase
  - Step 6.1: Prepare the Training Data (fill missing values and scale)
  - Step 6.2: Splitting Input Vectors and Outputs / Labels of Training Data
  - Step 6.3: Train the Support Vector Classifier
  - Step 6.4: Save the Trained Model
- Step 7: Execute the Testing Phase
  - Step 7.1: Prepare Input Vectors and Outputs / Labels of Testing Data
  - Step 7.2: Load the Saved Model
  - Step 7.3: Evaluate the Machine Learning Model
  - Step 7.4: Calculate the Accuracy Score
  - Step 7.5: Detailed Evaluation Metrics (Precision, Recall, F1, Confusion Matrix)
- Step 8: Execute the Application Phase
  - Step 8.1: Take Input from User
  - Step 8.2: Convert User Input into Feature Vector
  - Step 8.3: Fill and Scale the Feature Vector
  - Step 8.4: Load the Saved Model
  - Step 8.5: Model Prediction
- Step 9: Execute the Feedback Phase
  - Step 9.1: Collect Feedback from Users and Domain Experts
  - Step 9.2: Make a List of Potential Improvements
  - Step 9.3: Improve the Model Based on Feedback
- Step 10: Deploy Your Model as a Public Web App

- Conclusion

 <h2 id="Introduction-and-Input-/-Output">Introduction and Input / Output</h2> **Definition:** A binary classification task that predicts whether a patient has heart disease (1) or not (0) from four medical attributes. **Dataset:** UCI Heart Disease dataset – **Cleveland database** (`processed_cleveland.csv`, 303 patients). Source: https://archive.ics.uci.edu/dataset/45/heart+disease **Why Cleveland?** The four supplied files (Cleveland, Hungarian, Switzerland, VA) are separate databases and were **not combined**. Cleveland is the only one that is almost complete (only 6 missing values in total). In the other three, the `ca` and `thal` columns are mostly missing (for example 291 of 294 `ca` values in Hungarian), so they cannot support attributes that are useful for prediction. <h3 id="Inputs-(exactly-four-attributes-%E2%80%93-selected-in-Step-4)">Inputs (exactly four attributes – selected in Step 4)</h3> <thead> <th><tbody></tbody>

thal	Thallium stress-test result	3 = Normal, 6 = Fixed defect, 7 = Reversible defect
ca	Number of major blood vessels (0–3) coloured by fluoroscopy	0, 1, 2, 3
thalach	Maximum heart rate achieved (beats per minute)	Number (about 70 – 200)
oldpeak	ST depression caused by exercise (ECG measure)	Number (0.0 – 6.2)

<h3 id="Output-(Prediction)">Output (Prediction)</h3> <thead> <th><tbody> </tbody>

target	0 = No heart disease, 1 = Heart disease

(The four inputs above are confirmed by the feature-selection results in Step 4.)

 <h3 id="Step-1:-Import-Libraries">Step 1: Import Libraries</h3>

In [1]:
```
"""
Purpose of this code:
---------------------
Imports the libraries needed for loading data, selecting features, scaling,
training a Support Vector Classifier, evaluating it and saving it.
"""

# Numerical computing and data handling
import numpy as np
import pandas as pd

# Save and load Python objects (model, scaler, ...)
import pickle

# Split data into training and testing sets
from sklearn.model_selection import train_test_split

# Fill missing values and scale features
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
```

```
# Feature selection helpers
from sklearn.feature_selection import f_classif, mutual_info_classif
from sklearn.ensemble import RandomForestClassifier

# Support Vector Machine (SVM) model
from sklearn import svm

# Evaluation metrics
from sklearn.metrics import accuracy_score, classification_report, confusion_mat

# Clean table display
from prettytable import PrettyTable

# Plotting (confusion matrix heatmap)
import matplotlib.pyplot as plt
import seaborn as sns
```

 <h3 id="Step-2:-Load-Sample-Data">Step 2: Load Sample Data</h3> processed_cleveland.csv is the official UCI *processed* file. It has **no header row**, 14 columns, and uses ? for missing values. So the code below assigns the official UCI column names and converts ? into a missing value (NaN). <thead> <th>Column</th> <th>Meaning</th> <th>Column</th> <th>Meaning</th> </thead> <tbody> </tbody> <tbody></tbody>

age	Age in years	thalach	Maximum heart rate achieved
sex	1 = male, 0 = female	exang	Exercise-induced angina (1 = yes, 0 = no)
cp	Chest pain type (1–4)	oldpeak	ST depression induced by exercise
trestbps	Resting blood pressure (mm Hg)	slope	Slope of the peak exercise ST segment (1–3)
chol	Serum cholesterol (mg/dl)	ca	Major vessels coloured by fluoroscopy (0–3)
fbs	Fasting blood sugar > 120 mg/dl (1 = true)	thal	3 = normal, 6 = fixed defect, 7 = reversible defect
restecg	Resting ECG result (0–2)	num	Diagnosis of heart disease (0 – 4)

In [2]:
```
"""
Purpose of this section:
-------------------------
Loads the Cleveland heart disease data into a pandas DataFrame, using the
official UCI column names, and shows the first rows.
"""

# Official UCI attribute names (the file has no header row)
column_names = ["age", "sex", "cp", "trestbps", "chol", "fbs", "restecg",
                "thalach", "exang", "oldpeak", "slope", "ca", "thal", "num"]

# '?' in the file means "missing value"
sample_data = pd.read_csv("processed_cleveland.csv", header=None,
                          names=column_names, na_values="?")

print("\n\nSample Data (first 10 rows):")
print("============================\n")
```

```
pd.set_option("display.max_columns", None)
print(sample_data.head(10))
```

```
Sample Data (first 10 rows):
============================

    age  sex   cp  trestbps   chol  fbs  restecg  thalach  exang  oldpeak  \
0  63.0  1.0  1.0     145.0  233.0  1.0      2.0    150.0    0.0      2.3
1  67.0  1.0  4.0     160.0  286.0  0.0      2.0    108.0    1.0      1.5
2  67.0  1.0  4.0     120.0  229.0  0.0      2.0    129.0    1.0      2.6
3  37.0  1.0  3.0     130.0  250.0  0.0      0.0    187.0    0.0      3.5
4  41.0  0.0  2.0     130.0  204.0  0.0      2.0    172.0    0.0      1.4
5  56.0  1.0  2.0     120.0  236.0  0.0      0.0    178.0    0.0      0.8
6  62.0  0.0  4.0     140.0  268.0  0.0      2.0    160.0    0.0      3.6
7  57.0  0.0  4.0     120.0  354.0  0.0      0.0    163.0    1.0      0.6
8  63.0  1.0  4.0     130.0  254.0  0.0      2.0    147.0    0.0      1.4
9  53.0  1.0  4.0     140.0  203.0  1.0      2.0    155.0    1.0      3.1

   slope   ca  thal  num
0    3.0  0.0   6.0    0
1    2.0  3.0   3.0    2
2    2.0  2.0   7.0    1
3    3.0  0.0   3.0    0
4    1.0  0.0   3.0    0
5    1.0  0.0   3.0    0
6    3.0  2.0   3.0    3
7    1.0  0.0   3.0    0
8    2.0  1.0   7.0    2
9    3.0  0.0   7.0    1
```

 <h3 id="Step-3:-Understand-and-Pre-process-Sample-Data">Step 3: Understand and Pre-process Sample Data</h3><h4 id="Step-3.1:-Understand-Sample-Data">Step 3.1: Understand Sample Data</h4>

In [3]:
```
"""
Purpose of this section:
------------------------
Shows the attribute names, the number of instances, the data types,
the missing values and the original diagnosis (num) distribution.
"""

print("\n\nAttributes in Sample Data:")
print("==========================\n")
print(sample_data.columns)

print("\n\nNumber of Instances in Sample Data:", sample_data["num"].count())
print("=========================================\n")

print("Data types:")
print(sample_data.dtypes)

print("\n\nMissing values per attribute:")
print("=============================\n")
missing = sample_data.isnull().sum()
print(missing[missing > 0])
print("Total missing values:", int(missing.sum()))

print("\n\nOriginal diagnosis (num) distribution:")
```

```
print("=====================================\n")
print(sample_data["num"].value_counts().sort_index())
```

```
Attributes in Sample Data:
==========================

Index(['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach',
       'exang', 'oldpeak', 'slope', 'ca', 'thal', 'num'],
      dtype='str')


Number of Instances in Sample Data: 303
=======================================

Data types:
age         float64
sex         float64
cp          float64
trestbps    float64
chol        float64
fbs         float64
restecg     float64
thalach     float64
exang       float64
oldpeak     float64
slope       float64
ca          float64
thal        float64
num           int64
dtype: object


Missing values per attribute:
=============================

ca      4
thal    2
dtype: int64
Total missing values: 6


Original diagnosis (num) distribution:
======================================

num
0    164
1     55
2     36
3     35
4     13
Name: count, dtype: int64
```

What we see: all columns are already numbers (categorical attributes such as `cp`, `thal`, `slope` are stored as numeric codes). Only `ca` (4) and `thal` (2) have missing values. The diagnosis `num` has five levels (0 to 4).

<h4 id="Step-3.2:-Pre-process-Sample-Data">Step 3.2: Pre-process Sample Data</h4>

Two things are done here:

1. **Prepare the output.** `num = 0` means no disease and `num = 1, 2, 3, 4` mean disease is present (different severities). We convert it to a **binary** target: `0` = no disease, `1` = disease (values 1–4).
2. **Missing values.** We do **not** delete the 6 incomplete patients. They are filled later (Step 6.1) using values learned from the training data only, so no information leaks from the test data.

In [4]:
```
"""
Purpose of this section:
-------------------------
Creates the binary output attribute "target" from "num" and removes "num".
The prepared data is saved as a separate CSV file (the original file is untouche
"""

# 0 -> 0 (no disease);  1, 2, 3, 4 -> 1 (disease)
sample_data["target"] = (sample_data["num"] > 0).astype(int)
sample_data = sample_data.drop(columns=["num"])

print("Binary target distribution (0 = no disease, 1 = disease):")
print(sample_data["target"].value_counts().sort_index())

sample_data.to_csv("heart-disease-prepared.csv", index=False, header=True)
print("\nSaved: heart-disease-prepared.csv")
```

```
Binary target distribution (0 = no disease, 1 = disease):
target
0    164
1    139
Name: count, dtype: int64

Saved: heart-disease-prepared.csv
```

<h4 id="Step-3.3:-Split-Sample-Data-into-Training-Data-and-Testing-Data">Step 3.3: Split Sample Data into Training Data and Testing Data</h4> The split is done **now (before feature selection)** so the test data is never used to choose features or fill values. 80% is used for training and 20% for testing. `stratify` keeps the same disease / no-disease ratio in both parts.

In [5]:
```
"""
Purpose of this section:
-------------------------
Splits the data into Training Data (80%) and Testing Data (20%).
random_state=0 gives the same split every time.
"""

training_data, testing_data = train_test_split(
    sample_data,
    test_size=0.2,
    random_state=0,
    shuffle=True,
    stratify=sample_data["target"]
)

training_data.to_csv("training-data.csv", index=False, header=True)
testing_data.to_csv("testing-data.csv", index=False, header=True)

print("Training Data shape:", training_data.shape)
```

```
print("Testing Data shape:", testing_data.shape)
print("\nTraining target counts:\n", training_data["target"].value_counts().sort
print("\nTesting target counts:\n", testing_data["target"].value_counts().sort_i
```

```
Training Data shape: (242, 14)
Testing Data shape: (61, 14)

Training target counts:
 target
0    131
1    111
Name: count, dtype: int64

Testing target counts:
 target
0    33
1    28
Name: count, dtype: int64
```

 <h3 id="Step-4:-Feature-Selection-(exactly-4-input-attributes)">Step 4: Feature Selection (exactly 4 input attributes)</h3> The assignment allows only **4 input attributes**. Instead of choosing them by guess, every attribute is scored on the **training data only** using three simple methods:

- **ANOVA F-score** – how different the attribute values are between the two classes.
- **Mutual information** – how much the attribute tells us about the target.
- **Random Forest importance** – how much a Random Forest uses the attribute.

Each method ranks the 13 attributes (1 = best). The **average rank** decides the winners. If two attributes have the same average rank, the higher F-score wins. <h4 id="Step-4.1:-Rank-All-Attributes-using-Training-Data-only">Step 4.1: Rank All Attributes using Training Data only</h4>

In [6]:
```
"""
Purpose of this section:
-------------------------
Ranks the 13 candidate input attributes using the TRAINING data only.
(A temporary median fill is used here just so the ranking methods can run.)
"""

candidate_features = [c for c in training_data.columns if c != "target"]

X_rank = training_data[candidate_features]
y_rank = training_data["target"]

# Temporary fill of the few missing values (training data only)
X_rank_filled = pd.DataFrame(
    SimpleImputer(strategy="median").fit_transform(X_rank),
    columns=candidate_features
)

# Attributes that are really categories (codes), for mutual information
categorical_codes = ["sex", "cp", "fbs", "restecg", "exang", "slope", "ca", "tha
is_discrete = [c in categorical_codes for c in candidate_features]

# Method 1: ANOVA F-score
f_scores, _ = f_classif(X_rank_filled, y_rank)
```

```
# Method 2: Mutual information
mi_scores = mutual_info_classif(X_rank_filled, y_rank, discrete_features=is_disc

# Method 3: Random Forest importance
rf = RandomForestClassifier(n_estimators=300, random_state=0).fit(X_rank_filled,

ranking = pd.DataFrame({
    "F_score": f_scores,
    "Mutual_info": mi_scores,
    "RF_importance": rf.feature_importances_
}, index=candidate_features)

# Rank 1 = best for each method, then average the three ranks
ranking["rank_F"] = ranking["F_score"].rank(ascending=False)
ranking["rank_MI"] = ranking["Mutual_info"].rank(ascending=False)
ranking["rank_RF"] = ranking["RF_importance"].rank(ascending=False)
ranking["average_rank"] = ranking[["rank_F", "rank_MI", "rank_RF"]].mean(axis=1)

# Sort: best average rank first; ties are broken by the higher F-score
ranking = ranking.sort_values(["average_rank", "F_score"], ascending=[True, Fals

print("\n\nAttribute Ranking (Training Data only):")
print("=======================================\n")
print(ranking.round(3))
```

```
Attribute Ranking (Training Data only):
=======================================

          F_score  Mutual_info  RF_importance  rank_F  rank_MI  rank_RF  \
thal       84.619        0.136          0.125     1.0      1.0      1.0
ca         69.280        0.133          0.122     2.0      2.0      3.0
thalach    52.418        0.087          0.123     3.0      5.0      2.0
oldpeak    52.351        0.076          0.110     4.0      7.0      4.0
cp         39.802        0.119          0.088     6.0      3.0      6.0
exang      49.487        0.088          0.055     5.0      4.0     10.0
slope      34.458        0.079          0.055     7.0      6.0      9.0
age        15.386        0.071          0.098     9.0      8.0      5.0
sex        19.408        0.039          0.035     8.0      9.0     11.0
chol        2.994        0.024          0.083    12.0     11.0      7.0
restecg    10.290        0.030          0.022    10.0     10.0     12.0
trestbps    5.166        0.000          0.074    11.0     13.0      8.0
fbs         0.105        0.000          0.009    13.0     12.0     13.0

          average_rank
thal             1.000
ca               2.333
thalach          3.333
oldpeak          5.000
cp               5.000
exang            6.333
slope            7.333
age              7.333
sex              9.333
chol            10.000
restecg         10.667
trestbps        10.667
fbs             12.667
```

<h4 id="Step-4.2:-Select-the-Four-Attributes">Step 4.2: Select the Four Attributes</h4>

In [7]:
```
"""
Purpose of this section:
------------------------
Takes the top 4 attributes from the ranking. This list (and its ORDER)
is used for training, testing and deployment.
"""

selected_features = ranking.head(4).index.tolist()

meaning = {
    "age": "Age in years",
    "sex": "Sex (1 = male, 0 = female)",
    "cp": "Chest pain type (1-4)",
    "trestbps": "Resting blood pressure",
    "chol": "Serum cholesterol",
    "fbs": "Fasting blood sugar > 120 mg/dl",
    "restecg": "Resting ECG result",
    "thalach": "Maximum heart rate achieved during the stress test",
    "exang": "Exercise-induced angina",
    "oldpeak": "ST depression caused by exercise (ECG measure)",
    "slope": "Slope of the peak exercise ST segment",
    "ca": "Number of major vessels coloured by fluoroscopy (0-3)",
    "thal": "Thallium stress-test result (3 normal, 6 fixed defect, 7 reversible
}

assert len(selected_features) == 4, "Exactly 4 input attributes are required"

print("\n\nSelected Input Attributes (in this exact order):")
print("================================================\n")
for i, name in enumerate(selected_features, 1):
    print(f"{i}. {name:8s} - {meaning[name]}")
```

```
Selected Input Attributes (in this exact order):
================================================

1. thal     - Thallium stress-test result (3 normal, 6 fixed defect, 7 reversible
defect)
2. ca       - Number of major vessels coloured by fluoroscopy (0-3)
3. thalach  - Maximum heart rate achieved during the stress test
4. oldpeak  - ST depression caused by exercise (ECG measure)
```

Why these four? They have the best average rank across all three methods. They are also medically meaningful: they come from the heart stress test and blood-vessel imaging, which doctors use to check for heart disease. These are results on this dataset's training data; we do not claim they are the objectively best four attributes in general.

 <h3 id="Step-5:-Numeric-Representation-of-Input-and-Output-(No-Label-Encoding)">Step 5: Numeric Representation of Input and Output (No Label Encoding) </h3> In the Titanic notebook, the values were words (`Male`, `First`, ...), so they had to be converted to numbers. **The heart disease data is already numeric**: categorical attributes are stored as official UCI numeric codes (for example `thal` = 3, 6 or 7). So, as instructed, **no label encoding is used** in this notebook. The existing numeric codes are used as they are. <h4 id="Step-5.1:-Check-the-Output">Step 5.1: Check the Output</h4>

In [8]:
```
"""
Purpose of this section:
------------------------
Confirms the output "target" is numeric (0 or 1).
"""

print("Output attribute: target")
print("Data type:", training_data["target"].dtype)
print("Values present:", sorted(training_data["target"].unique()))
print("Meaning: 0 = No heart disease, 1 = Heart disease")
```

```
Output attribute: target
Data type: int64
Values present: [np.int64(0), np.int64(1)]
Meaning: 0 = No heart disease, 1 = Heart disease
```

<h4 id="Step-5.2:-Check-the-Input">Step 5.2: Check the Input</h4>

In [9]:
```
"""
Purpose of this section:
------------------------
Confirms the four selected input attributes are numeric and shows the
range of values / codes in the training data.
"""

print("Selected input attributes:\n")
for name in selected_features:
    col = training_data[name]
    print(f"{name:8s} type = {str(col.dtype):8s} min = {col.min()}  max = {col.m
          f"distinct values = {col.nunique()}  missing = {int(col.isnull().sum()
```

```
Selected input attributes:

thal     type = float64  min = 3.0  max = 7.0  distinct values = 3  missing = 1
ca       type = float64  min = 0.0  max = 3.0  distinct values = 4  missing = 4
thalach  type = float64  min = 71.0  max = 195.0  distinct values = 83  missing =
0
oldpeak  type = float64  min = 0.0  max = 6.2  distinct values = 37  missing = 0
```

 <h3 id="Step-6:-Execute-the-Training-Phase">Step 6: Execute the Training Phase</h3><h4 id="Step-6.1:-Prepare-the-Training-Data-(fill-missing-values-and-scale)">Step 6.1: Prepare the Training Data (fill missing values and scale)</h4>

- **Fill missing values:** the median of each selected attribute in the **training data** is used (no patient is deleted).
- **Scale:** `StandardScaler` puts the attributes on a similar scale (important for the SVM). It is **fitted on the training data only**, and the same fitted scaler is later applied to the test data and to new patients.

In [10]:
```
"""
Purpose of this section:
------------------------
Keeps only the 4 selected attributes, fills missing values and scales them.
Both objects are FITTED on training data only (prevents data leakage).
"""

training_inputs = training_data[selected_features]
```

```
imputer = SimpleImputer(strategy="median")
imputer.fit(training_inputs)
training_inputs_filled = imputer.transform(training_inputs)

scaler = StandardScaler()
scaler.fit(training_inputs_filled)
training_inputs_scaled = scaler.transform(training_inputs_filled)

training_data_prepared = pd.DataFrame(training_inputs_scaled, columns=selected_f
                                      index=training_data.index)
training_data_prepared["target"] = training_data["target"]

training_data_prepared.to_csv("training-data-prepared.csv", index=False, header=
print("Training data after filling and scaling (first 5 rows):\n")
print(training_data_prepared.head())
print("\nSaved: training-data-prepared.csv")
```

```
Training data after filling and scaling (first 5 rows):

        thal        ca   thalach   oldpeak  target
111 -0.86224  0.342066 -0.197216  0.102190       1
52  -0.86224  0.342066  0.192210 -0.900375       1
173 -0.86224 -0.719215  0.365288  0.102190       0
138  1.21400 -0.719215 -0.802991  0.436378       1
11  -0.86224 -0.719215  0.192210  0.185737       0

Saved: training-data-prepared.csv
```

<h4 id="Step-6.2:-Splitting-Input-Vectors-and-Outputs-/-Labels-of-Training-Data">Step 6.2: Splitting Input Vectors and Outputs / Labels of Training Data</h4>

In [11]:
```
"""
Purpose of this section:
------------------------
Separates the input vectors (4 selected attributes) from the output labels.
The target is never part of the inputs.
"""

input_vector_train = training_data_prepared[selected_features]
output_label_train = training_data_prepared["target"]

print("Input vectors (train) shape:", input_vector_train.shape)
print("Output labels (train) shape:", output_label_train.shape)
```

```
Input vectors (train) shape: (242, 4)
Output labels (train) shape: (242,)
```

<h4 id="Step-6.3:-Train-the-Support-Vector-Classifier">Step 6.3: Train the Support Vector Classifier</h4>

In [12]:
```
"""
Purpose of this section:
------------------------
Trains a Support Vector Classifier (SVC) - the same model as the Titanic
notebook - on the training data (4 inputs -> heart disease yes / no).
"""

print("\n\nTraining the Support Vector Classifier on Training Data")
print("=========================================================\n")
```

```
svc_model = svm.SVC(gamma="auto", random_state=0)
svc_model.fit(input_vector_train, np.ravel(output_label_train))

print(svc_model)
```

```
Training the Support Vector Classifier on Training Data
=======================================================

SVC(gamma='auto', random_state=0)
```

<h4 id="Step-6.4:-Save-the-Trained-Model">Step 6.4: Save the Trained Model</h4>
Besides the model, the imputer, the scaler and the **list of the four feature names (in order)** are saved. The web app loads exactly these files.

In [13]:
```
"""
Purpose of this section:
------------------------
Saves the trained model and everything needed to prepare new data
in exactly the same way.
"""

pickle.dump(svc_model, open("heart_disease_model.pkl", "wb"))
pickle.dump(imputer, open("imputer.pkl", "wb"))
pickle.dump(scaler, open("scaler.pkl", "wb"))
pickle.dump(selected_features, open("selected_features.pkl", "wb"))

print("Saved: heart_disease_model.pkl")
print("Saved: imputer.pkl")
print("Saved: scaler.pkl")
print("Saved: selected_features.pkl ->", selected_features)
```

```
Saved: heart_disease_model.pkl
Saved: imputer.pkl
Saved: scaler.pkl
Saved: selected_features.pkl -> ['thal', 'ca', 'thalach', 'oldpeak']
```

 <h3 id="Step-7:-Execute-the-Testing-Phase">Step 7: Execute the Testing Phase</h3><h4 id="Step-7.1:-Prepare-Input-Vectors-and-Outputs-/-Labels-of-Testing-Data">Step 7.1: Prepare Input Vectors and Outputs / Labels of Testing Data</h4>
The test data is prepared with the imputer and scaler **already fitted on the training data** (`transform` only - no new fitting).

In [14]:
```
"""
Purpose of this section:
------------------------
Applies the fitted imputer and scaler to the testing data and separates
the input vectors from the output labels.
"""

testing_inputs = testing_data[selected_features]
testing_inputs_scaled = scaler.transform(imputer.transform(testing_inputs))

input_vector_test = pd.DataFrame(testing_inputs_scaled, columns=selected_feature
                                 index=testing_data.index)
output_label_test = testing_data["target"]

print("Input vectors (test) shape:", input_vector_test.shape)
print("Output labels (test) shape:", output_label_test.shape)
```

```
Input vectors (test) shape: (61, 4)
Output labels (test) shape: (61,)
```

<h4 id="Step-7.2:-Load-the-Saved-Model">Step 7.2: Load the Saved Model</h4>

In [15]:
```
"""
Purpose of this section:
------------------------
Loads the saved model from disk into memory.
"""

model = pickle.load(open("heart_disease_model.pkl", "rb"))
print("Model loaded from heart_disease_model.pkl")
```

```
Model loaded from heart_disease_model.pkl
```

<h4 id="Step-7.3:-Evaluate-the-Machine-Learning-Model">Step 7.3: Evaluate the Machine Learning Model</h4><h5 id="Step-7.3.1:-Make-Predictions-with-the-Trained-Model-on-Testing-Data">Step 7.3.1: Make Predictions with the Trained Model on Testing Data</h5>

In [16]:
```
"""
Purpose of this section:
------------------------
Predicts heart disease for every patient in the testing data and puts
the actual and predicted values side by side.
"""

model_predictions = model.predict(input_vector_test)

model_predictions = pd.DataFrame({
    "Actual": output_label_test.values,
    "Predictions": model_predictions
}, index=testing_data.index)

model_predictions.to_csv("model-predictions.csv", index=False, header=True)

print("\n\nPredictions Returned by heart_disease_model (first 15 test patients):
print("=====================================================================\n")
print(model_predictions.head(15))
```

```
Predictions Returned by heart_disease_model (first 15 test patients):
======================================================================

     Actual  Predictions
125       0            0
83        1            1
53        0            0
45        1            1
248       1            1
211       1            0
36        1            1
106       1            1
120       1            1
31        1            1
156       1            0
157       1            1
148       0            0
253       0            0
250       0            1
```

<h4 id="Step-7.4:-Calculate-the-Accuracy-Score">Step 7.4: Calculate the Accuracy Score</h4>

In [17]:
```
"""
Purpose of this section:
------------------------
Calculates how accurate the model is on the testing data (and, for
comparison, on the training data).
"""

model_accuracy_score = accuracy_score(model_predictions["Actual"], model_predict
train_accuracy_score = accuracy_score(output_label_train, model.predict(input_ve

print("\n\nAccuracy Score:")
print("===============\n")
print("Testing accuracy :", round(model_accuracy_score, 2))
print("Training accuracy:", round(train_accuracy_score, 2))
```

```
Accuracy Score:
===============

Testing accuracy : 0.84
Training accuracy: 0.82
```

<h4 id="Step-7.5:-Detailed-Evaluation-Metrics-(Precision,-Recall,-F1,-Confusion-Matrix)">Step 7.5: Detailed Evaluation Metrics (Precision, Recall, F1, Confusion Matrix)</h4>

- **Precision:** of the patients predicted as "Heart disease", how many really have it?
- **Recall:** of the patients who really have heart disease, how many did the model find? (Very important in medicine - a missed patient is costly.)
- **F1-score:** balance of precision and recall.
- **Confusion matrix:** a table of correct and wrong predictions for each class.

In [18]:
```
"""
Purpose of this section:
```

```
-------------------------
Prints precision, recall, F1-score and the confusion matrix (table + heatmap).
"""

class_names = ["No Heart Disease (0)", "Heart Disease (1)"]

print("\nClassification Report:")
print("======================\n")
print(classification_report(model_predictions["Actual"], model_predictions["Pred
                            target_names=class_names, zero_division=0))

conf_matrix = confusion_matrix(model_predictions["Actual"], model_predictions["P
conf_matrix_df = pd.DataFrame(
    conf_matrix,
    index=["Actual: No Disease", "Actual: Disease"],
    columns=["Predicted: No Disease", "Predicted: Disease"]
)
print("\nConfusion Matrix:")
print("=================\n")
print(conf_matrix_df)

plt.figure(figsize=(5, 4))
sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="Blues",
            xticklabels=["No Disease", "Disease"],
            yticklabels=["No Disease", "Disease"])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - SVC on Heart Disease Testing Data")
plt.tight_layout()
plt.show()
```

```
Classification Report:
======================

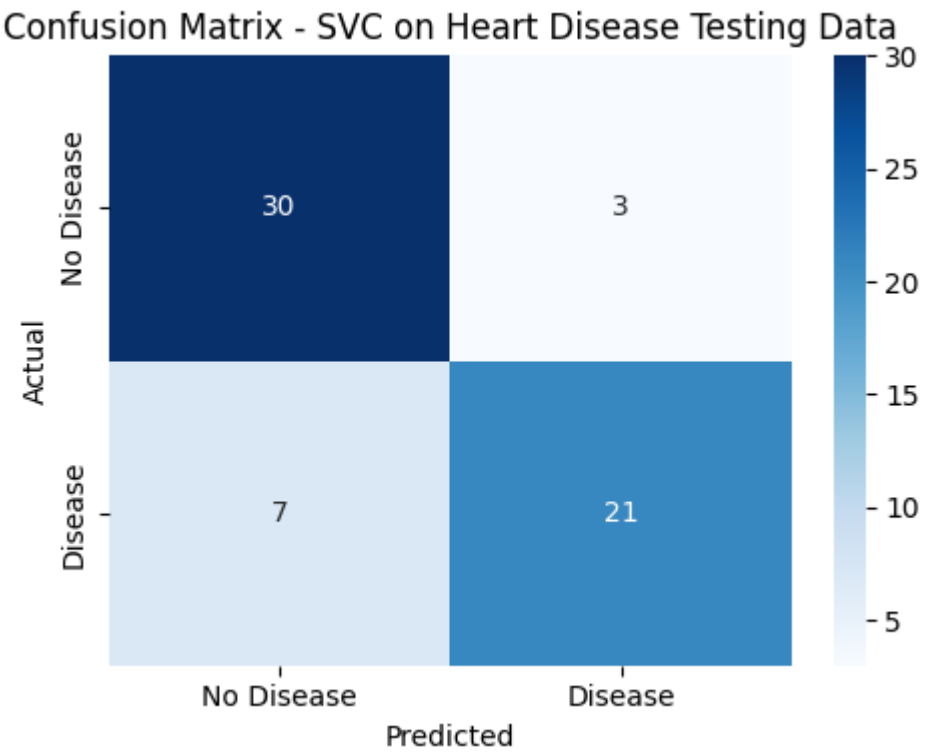
                     precision    recall  f1-score   support

No Heart Disease (0)       0.81      0.91      0.86        33
   Heart Disease (1)       0.88      0.75      0.81        28

            accuracy                           0.84        61
           macro avg       0.84      0.83      0.83        61
        weighted avg       0.84      0.84      0.83        61


Confusion Matrix:
=================

                    Predicted: No Disease  Predicted: Disease
Actual: No Disease                     30                   3
Actual: Disease                         7                  21
```



In [19]:
```
"""
Purpose of this section:
------------------------
Explains the result above in simple words (numbers come from the run above).
"""

tn, fp, fn, tp = conf_matrix.ravel()
total = tn + fp + fn + tp

print(f"The model was tested on {total} patients it had never seen.")
print(f" - {tp + tn} predictions were correct ({tp} disease cases and {tn} healt
print(f" - {fn} patients WITH heart disease were missed (predicted as healthy)."
print(f" - {fp} healthy patients were wrongly flagged as having heart disease.")
```

```
The model was tested on 61 patients it had never seen.
 - 51 predictions were correct (21 disease cases and 30 healthy cases).
 - 7 patients WITH heart disease were missed (predicted as healthy).
 - 3 healthy patients were wrongly flagged as having heart disease.
```

Limitations: the dataset is small (303 patients, only 61 in the test set), so the scores can change noticeably with a different split. The model uses only four attributes (an assignment restriction). This is a learning project, **not a medical tool**.

 <h3 id="Step-8:-Execute-the-Application-Phase">Step 8: Execute the Application Phase</h3><h4 id="Step-8.1:-Take-Input-from-User">Step 8.1: Take Input from User</h4>

In [20]:
```
"""
Purpose of this section:
------------------------
Takes the four inputs for ONE new patient. A demo patient is pre-filled so
the notebook runs top-to-bottom without typing. To enter your own values,
replace the four demo lines with:

    thal_input    = float(input("Enter thal (3 = Normal, 6 = Fixed defect, 7 = R
```

```
    ca_input      = float(input("Enter ca - number of major vessels (0, 1, 2, 3)
    thalach_input = float(input("Enter thalach - maximum heart rate (60 to 220):
    oldpeak_input = float(input("Enter oldpeak - ST depression (0.0 to 7.0): "))
"""

# Demo patient (edit these values)
thal_input = 7
ca_input = 2
thalach_input = 130
oldpeak_input = 2.0

# Check that the values are valid before using them
if thal_input not in (3, 6, 7):
    raise ValueError("thal must be 3, 6 or 7")
if ca_input not in (0, 1, 2, 3):
    raise ValueError("ca must be 0, 1, 2 or 3")
if not (60 <= thalach_input <= 220):
    raise ValueError("thalach must be between 60 and 220")
if not (0 <= oldpeak_input <= 7):
    raise ValueError("oldpeak must be between 0 and 7")

print(f"thal: {thal_input}, ca: {ca_input}, thalach: {thalach_input}, oldpeak: {
```

```
thal: 7, ca: 2, thalach: 130, oldpeak: 2.0
```

<h4 id="Step-8.2:-Convert-User-Input-into-Feature-Vector-(Exactly-Same-as-Feature-Vectors-of-Sample-Data)">Step 8.2: Convert User Input into Feature Vector (Exactly Same as Feature Vectors of Sample Data)</h4>

In [21]:

```
"""
Purpose of this section:
-------------------------
Puts the user input into a DataFrame with the same four columns,
in the same order as training (taken from the saved feature list).
"""

saved_features = pickle.load(open("selected_features.pkl", "rb"))

user_values = {"thal": thal_input, "ca": ca_input,
               "thalach": thalach_input, "oldpeak": oldpeak_input}

user_input = pd.DataFrame([user_values])[saved_features]

print("\n\nUser Input Feature Vector:")
print("==========================\n")
print(user_input)
```

```
User Input Feature Vector:
==========================

   thal  ca  thalach  oldpeak
0     7   2      130      2.0
```

<h4 id="Step-8.3:-Fill-and-Scale-the-Feature-Vector-(Exactly-Same-as-the-Training-Data)">Step 8.3: Fill and Scale the Feature Vector (Exactly Same as the Training Data)</h4> This step replaces the label encoding step of the Titanic notebook. The saved imputer and scaler are loaded and applied (<code>transform</code> only).

In [22]:
```
"""
Purpose of this section:
------------------------
Applies the SAVED imputer and scaler to the user's feature vector.
"""

saved_imputer = pickle.load(open("imputer.pkl", "rb"))
saved_scaler = pickle.load(open("scaler.pkl", "rb"))

unseen_data_features = saved_scaler.transform(saved_imputer.transform(user_input
unseen_data_features = pd.DataFrame(unseen_data_features, columns=saved_features

print("\n\nUser Input Scaled Feature Vector:")
print("==================================\n")
print(unseen_data_features)
```

```
User Input Scaled Feature Vector:
==================================

    thal        ca   thalach   oldpeak
0  1.214  1.403347 -0.802991  0.770566
```

<h4 id="Step-8.4:-Load-the-Saved-Model">Step 8.4: Load the Saved Model</h4>

In [23]:
```
"""
Purpose of this section:
------------------------
Loads the saved model for the Application Phase.
"""

model = pickle.load(open("heart_disease_model.pkl", "rb"))
```

<h4 id="Step-8.5:-Model-Prediction">Step 8.5: Model Prediction</h4><h5 id="Step-8.5.1:-Apply-Model-on-the-Feature-Vector-of-the-unseen-patient-and-return-Prediction-to-the-User">Step 8.5.1: Apply Model on the Feature Vector of the unseen patient and return Prediction to the User</h5>

In [24]:
```
"""
Purpose of this section:
------------------------
Uses the trained model to predict heart disease for the new patient.
"""

predicted_disease = model.predict(unseen_data_features)[0]

if predicted_disease == 1:
    prediction = "HEART DISEASE PREDICTED (1)"
else:
    prediction = "NO HEART DISEASE PREDICTED (0)"

pretty_table = PrettyTable()
pretty_table.add_column("   ** Model Prediction of Heart Disease **   ", [predic

print(pretty_table)
```

```
+-----------------------------------------------+
|    ** Model Prediction of Heart Disease **    |
+-----------------------------------------------+
|          HEART DISEASE PREDICTED (1)          |
+-----------------------------------------------+
```

 <h3 id="Step-9:-Execute-the-Feedback-Phase">Step 9: Execute the Feedback Phase</h3><h4 id="Step-9.1:-Collect-Feedback-from-Users-and-Domain-Experts-on-Performance-of-the-Model-Deployed-in-the-Real-World">Step 9.1: Collect Feedback from Users and Domain Experts on Performance of the Model Deployed in the Real World</h4> In a real project, the live app would be shown to **cardiologists / doctors** (domain experts) and to **test users**. They would check whether the predictions agree with real diagnoses, whether the four inputs are easy to enter, and which cases the model gets wrong. *(No real feedback has been collected for this assignment, so none is invented here.)* <h4 id="Step-9.2:-Make-a-List-of-Potential-Improvements">Step 9.2: Make a List of Potential Improvements</h4>

In [25]:
```
"""
Purpose of this section:
------------------------
Potential improvements, based on the limitations seen in this notebook.
"""

improvements = [
    "Use more patients: the Cleveland data has only 303 rows, so results change
    "Use cross-validation instead of one 80/20 split to get a more stable accura
    "Compare other models (Logistic Regression, Decision Tree, Random Forest) an
    "Add more attributes (for example chest pain type, age) if the 4-attribute l
    "Reduce missed disease cases (higher recall), because missing a sick patient
    "Ask doctors to review the predictions before any real-world use",
]
for i, item in enumerate(improvements, 1):
    print(f"{i}. {item}")
```

```
1. Use more patients: the Cleveland data has only 303 rows, so results change wit
h the split
2. Use cross-validation instead of one 80/20 split to get a more stable accuracy
estimate
3. Compare other models (Logistic Regression, Decision Tree, Random Forest) and t
une settings using only training data
4. Add more attributes (for example chest pain type, age) if the 4-attribute limi
t is lifted
5. Reduce missed disease cases (higher recall), because missing a sick patient is
the costly mistake
6. Ask doctors to review the predictions before any real-world use
```

<h4 id="Step-9.3:-Improve-the-Model-Based-on-Feedback">Step 9.3: Improve the Model Based on Feedback</h4> In the next version, the improvements above would be applied, and Steps 6-7 would be re-run to confirm that the new model performs better **before** replacing the deployed model files.

 <h2 id="Step-10:-Deploy-Your-Model-as-a-Public-Web-App">Step 10: Deploy Your Model as a Public Web App</h2> **Files to upload to your GitHub repository (all in the same folder):** <thead> <th>File</th> <th>Purpose</th> </thead> <tbody> </tbody> <tbody></tbody>

`app.py`	The Streamlit web app (4 input fields -> prediction)
`requirements.txt`	Packages the cloud must install
`heart_disease_model.pkl`	Trained model (from Step 6.4)
`imputer.pkl` , `scaler.pkl`	Same preprocessing used in training
`selected_features.pkl`	The four feature names in the right order

Steps

1. Create a free GitHub account and a new **Public** repository (for example `heart-disease-app`).
2. Upload the six files above to the repository (**Add file -> Upload files**).
3. Go to https://share.streamlit.io and sign in with GitHub.
4. Click **Create app -> Deploy a public app from GitHub**, choose your repository, branch `main` and main file `app.py` , then click **Deploy**.
5. If the build fails with a `ModuleNotFoundError` or version error, open **Settings -> General -> Python version** and choose **3.11** or **3.12**, then reboot.
6. The streamlit app link is: (https://heart-disease-app-jes4zdm4hfczcb3vxypo7w.streamlit.app/).

Quick check before uploading: the next cell loads the saved files and makes a prediction exactly the way `app.py` does.

In [26]:
```
"""
Purpose of this section:
------------------------
Checks that all deployment files exist and that the saved files give a
valid prediction (same logic as app.py).
"""

import os

required_files = ["app.py", "requirements.txt", "heart_disease_model.pkl",
                  "imputer.pkl", "scaler.pkl", "selected_features.pkl"]

for f in required_files:
    print(f"{f:28s}", "found" if os.path.exists(f) else "MISSING - copy it into

# Test the saved files with the demo patient from Step 8
m = pickle.load(open("heart_disease_model.pkl", "rb"))
imp = pickle.load(open("imputer.pkl", "rb"))
sc = pickle.load(open("scaler.pkl", "rb"))
feats = pickle.load(open("selected_features.pkl", "rb"))

assert len(feats) == 4, "The app must use exactly 4 input features"
row = pd.DataFrame([{"thal": 7, "ca": 2, "thalach": 130, "oldpeak": 2.0}])[feats
print("\nFeatures used by the app:", feats)
row_scaled = pd.DataFrame(sc.transform(imp.transform(row)), columns=feats)
print("Prediction for the demo patient:", int(m.predict(row_scaled)[0]))
```

```
app.py                      found
requirements.txt            found
heart_disease_model.pkl     found
imputer.pkl                 found
scaler.pkl                  found
selected_features.pkl       found

Features used by the app: ['thal', 'ca', 'thalach', 'oldpeak']
Prediction for the demo patient: 1
```

 <h2 id="Conclusion">Conclusion</h2>

In [27]:
```
"""
Purpose of this section:
------------------------
Prints the final summary using the actual results of this run.
"""

print("Heart Disease Prediction System - Summary")
print("=========================================\n")
print("Dataset            : UCI Heart Disease - Cleveland database (303 patients
print("Input attributes   :", ", ".join(selected_features))
print("Output             : target (0 = no heart disease, 1 = heart disease)")
print("Model              : Support Vector Classifier (gamma='auto')")
print(f"Testing accuracy   : {model_accuracy_score:.2f}  (training accuracy: {tr
print(f"Disease cases found: {tp} of {tp + fn} (recall)")
print(f"Healthy flagged    : {fp} of {tn + fp}")
```

```
Heart Disease Prediction System - Summary
=========================================

Dataset            : UCI Heart Disease - Cleveland database (303 patients)
Input attributes   : thal, ca, thalach, oldpeak
Output             : target (0 = no heart disease, 1 = heart disease)
Model              : Support Vector Classifier (gamma='auto')
Testing accuracy   : 0.84  (training accuracy: 0.82)
Disease cases found: 21 of 28 (recall)
Healthy flagged    : 3 of 33
```

The four input attributes were selected using training data only, no label encoding is used, and the saved model, imputer, scaler and feature list are the same files used by the web app. Because the dataset is small, the results are only a learning example and not a medical diagnosis tool.